

Integración de Servicios Cloud GCP

Linux, terminal e interfaces de red en una máquina virtual

Semana 02 - Sesión 04

Rel Guzman



**Universidad
Tecnológica
del Perú**

Tres conceptos de la sesión anterior

Anfitrión: equipo físico que aporta CPU, memoria, disco y red.

Hipervisor: programa que crea y administra máquinas virtuales.

Máquina virtual (MV): computadora definida mediante software que recibe recursos del anfitrión.

Ejemplo completo

Anfitrión: laptop del estudiante.



Hipervisor: VirtualBox, VMware u otro disponible.



Sistema invitado: sistema operativo que inicia dentro de la MV.

Ubuntu Desktop

Sistema Linux preparado para usar con escritorio gráfico. Puede ejecutarse directamente en el equipo o dentro de una máquina virtual.

Configuración sugerida para la MV

4 GB de RAM · 2 núcleos asignados · 20 GB de disco · 1 dispositivo virtual de conexión con salida a internet.

Resultado completo esperado

- ✓ Ubuntu muestra el escritorio.
- ✓ La aplicación Terminal abre.
- ✓ Conoces la contraseña del usuario.
- ✓ La MV tiene conexión de red.

Logro de la sesión

Al finalizar, podrás usar la aplicación Terminal de Ubuntu para identificar el sistema, desplazarte entre carpetas, consultar la configuración básica de red y reutilizar instrucciones, comprobando el resultado de cada acción.

Sistema operativo Conjunto de programas que permite utilizar el hardware y ejecutar aplicaciones.

Kernel Núcleo del sistema operativo: coordina CPU, memoria, disco, red y dispositivos.

Unix Familia de sistemas operativos iniciada en 1969; popularizó usuarios, reglas de acceso (permisos), programas en ejecución (procesos) y un árbol único de archivos.

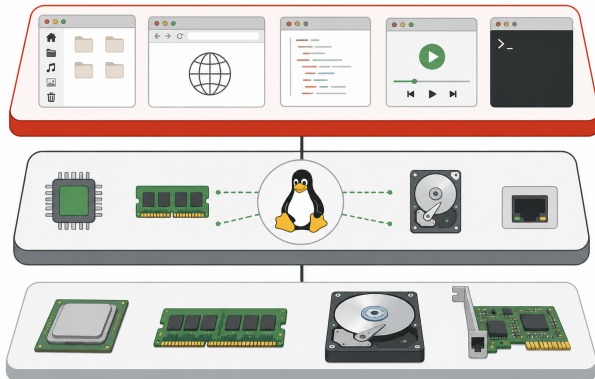
Linux Kernel libre creado en 1991 que sigue muchas ideas de Unix. Osea Unix no era libre.

Distribución Sistema listo para instalar: reúne Linux, aplicaciones, instalador, actualizaciones y configuración.

Ejemplo

Ubuntu es una **distribución**; utiliza el **kernel Linux** para administrar los recursos del equipo.

La distribución añade aplicaciones sobre el kernel



Arriba: Ubuntu agrega escritorio, archivos, navegador, aplicaciones y terminal.

Centro: Linux coordina esos recursos.

Abajo: hardware — CPU, memoria, disco y red.

Unix y Linux comparten ideas, no el mismo origen

Aspecto	Unix	Linux
Origen	Bell Labs, 1969	Linus Torvalds, 1991
Qué era al inicio	Sistema operativo completo	Kernel
Uso actual	El Unix original ya no se usa; existen sistemas derivados y sistemas que cumplen el estándar UNIX	Kernel de muchas distribuciones GNU/Linux
Ejemplos	AIX, HP-UX y macOS	Ubuntu, Debian, Mint y Kali usan Linux

Resultado de la comparación

Linux no es una versión de Unix: es otro kernel que adoptó muchas de sus ideas y formas de trabajo.

Dos definiciones antes del ejemplo

Un **paquete** es un archivo que contiene un programa y la información necesaria para instalarlo. Un **gestor de paquetes** instala, actualiza o elimina esos paquetes.

Familia	Distribuciones	Paquete y gestor
Debian	Debian, Ubuntu, Mint, Kali	.deb; apt y dpkg
Red Hat	RHEL, Fedora, Rocky, AlmaLinux	.rpm; dnf y rpm
Arch / SUSE	Arch, Manjaro, openSUSE	pacman / zypper

Resultado para el laboratorio: Ubuntu pertenece a la familia Debian; por eso trabajaremos con paquetes .deb y el gestor apt.

Definiciones necesarias

Un **archivo** .deb es el formato de paquete usado por Debian y Ubuntu. Los **metadatos** describen el paquete. Una **dependencia** es otro paquete necesario. La **arquitectura** indica el tipo de equipo compatible.



Así se ve el nombre del archivo

nano_7.2_amd64.deb

nano	nombre del programa
7.2	versión del paquete
amd64	arquitectura de 64 bits
.deb	formato de Debian y Ubuntu

Qué contiene

Metadatos —nombre, versión y dependencias—, archivos del programa y documentación.

Resultado: Ubuntu reconoce el archivo .deb porque pertenece a la familia Debian.

Punto de partida

La MV puede contener una instalación reciente de Ubuntu o un sistema que ya fue utilizado.

Índice local

Catálogo que Ubuntu guarda en la MV con los paquetes y versiones disponibles en los repositorios configurados.

1. Inicia la MV.
2. Espera hasta que aparezca el escritorio de Ubuntu.
3. Como primera tarea, actualiza el índice local.

Siguiente acción

La próxima diapositiva muestra la instrucción y el resultado esperado de la actualización.

Definiciones necesarias

Un **repositorio** es un servidor que almacena paquetes. El **índice local** es el catálogo de paquetes que Ubuntu guarda en la MV. `sudo` ejecuta una instrucción con permisos de administración.

Qué ocurre

1. Ejecutas `sudo apt update`.
2. `apt` consulta los repositorios configurados.
3. Ubuntu renueva su índice local de paquetes.

Ejemplo y resultado esperado

```
sudo apt update
Obj:1 archive.ubuntu.com ubuntu InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Todos los paquetes están actualizados.
```

Comprobación: las tareas de lectura y dependencias muestran `Hecho`. Las líneas pueden variar según Ubuntu y los repositorios configurados.

¿Por qué actualizar el índice antes de instalar?

Tres definiciones para el ejemplo

Node.js ejecuta JavaScript fuera del navegador. **npm** gestiona paquetes para proyectos Node.js. Una **API** es un servicio que recibe solicitudes y devuelve datos o resultados.

La razón

apt consulta el índice local para localizar versiones disponibles y sus dependencias. Al actualizarlo primero, evita buscar información antigua antes de una instalación.

Para una API

1. Actualiza el índice local.
2. Instala nodejs y npm.
3. Comprueba ambas versiones.

Resultado: la MV dispone del entorno y del gestor necesarios para preparar una API con Node.js.

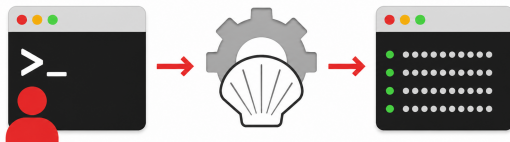
Instalación y resultado esperado

```
sudo apt install nodejs npm
¿Desea continuar? [S/n] s
Configurando nodejs (...) ...
Configurando npm (...) ...
node --version
v18.x.x
npm --version
9.x.x
```

Ejecutando comandos

Dos definiciones antes del ejemplo

La **Terminal** es la aplicación donde escribes instrucciones y lees resultados. La **shell** es el programa que interpreta cada instrucción y solicita su ejecución.



Resultado completo

whoami es el comando que solicita el nombre del usuario actual. La shell lo interpreta y la terminal muestra el resultado.

```
rgap@ubuntu:~$ whoami  
rgap  
rgap@ubuntu:~$
```

Una línea distingue cuatro elementos

El **prompt** aparece antes de escribir. La **instrucción** comienza con el comando y puede incluir una opción y un argumento.

<code>rgap@ubuntu:~\$</code>	<code>ls</code>	<code>-a</code>	<code>/etc</code>
Prompt Texto previo que identifica usuario, equipo y ubicación.	Comando Acción que se solicita: listar.	Opción Cambia cómo actúa el comando.	Argumento Dato sobre el que actúa: <code>/etc</code> .

Lectura completa

El prompt indica dónde estás. La instrucción `ls -a /etc` solicita el contenido de `/etc`, incluidos los elementos ocultos.

Sistema de archivos Forma en que Linux organiza directorios y archivos.

Directorio Contenedor que puede guardar archivos y otros directorios.

Raíz / Punto inicial del árbol completo.

Ruta absoluta Ubicación que comienza en /; no depende del directorio actual.

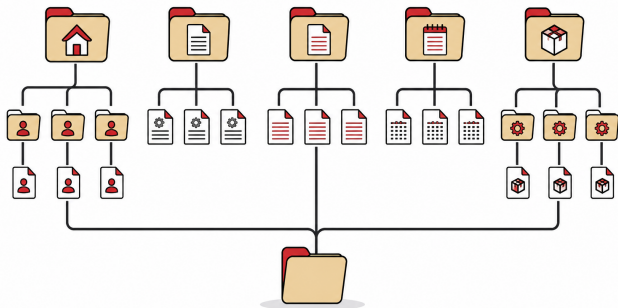
Ruta relativa Ubicación calculada desde el directorio actual.

Directorio personal Espacio del usuario, representado también por ~.

Ejemplo

Absoluta: `/home/rgap/Documentos` · relativa desde `/home/rgap`: `Documentos`

Cada rama del árbol cumple una función



Resultado completo

/home: archivos personales.

/etc: configuración.

/var: datos variables y registros de eventos (logs).

/tmp: archivos temporales.

/usr/bin: programas y comandos.

Todos descienden de la raíz /.

Definición

pwd muestra la ruta completa del directorio en el que estás trabajando.

Ejemplo y resultado completo

```
rgap@ubuntu:~$ pwd  
/home/rgap
```

Cómo comprobarlo

La salida comienza con /; por eso es una ruta absoluta. En este ejemplo, corresponde al directorio personal de rgap.

ls -a: muestra también los elementos ocultos

Dos definiciones

ls lista el contenido del directorio actual. La opción -a incluye los nombres ocultos. Un **repositorio Git** es un directorio cuyo historial y configuración se guardan en el directorio oculto .git.

Ejemplo en el directorio de una API

```
rgap@ubuntu:~/api-repo$ ls -a
.  ..  .git  .gitignore  README.md
package.json  src
```

Resultado

.git confirma que el directorio es un repositorio Git. . representa la ubicación actual y .., la ubicación superior.

Definición

`mkdir` crea un directorio nuevo dentro de la ubicación actual.

Ejemplo y comprobación

```
rgap@ubuntu:~$ mkdir api
rgap@ubuntu:~$ ls -a
.  ..  api
```

Resultado

`mkdir api` no imprime una confirmación cuando funciona. El resultado se comprueba porque `api` aparece en el listado.

cd: cambia la ubicación actual

Definición

cd cambia la ubicación actual al directorio indicado como argumento.

Ejemplo y resultado completo

```
rgap@ubuntu:~$ cd api
rgap@ubuntu:~/api$ pwd
/home/rgap/api
```

Cómo comprobarlo

El prompt cambia a ~/api y pwd confirma la nueva ruta completa.

Definición

cat lee un archivo y muestra su contenido como salida.

Ejemplo en un repositorio Git

```
rgap@ubuntu:~/api-repo$ cat README.md  
# API del curso  
Servicio de ejemplo con Node.js
```

Resultado

La salida coincide con el texto guardado en README.md; el archivo no se modifica.

`whoami` Usuario actual.
`hostname` Nombre del equipo.
`cat /etc/os-release` Distribución y versión.
`uname -a` Kernel, nombre del equipo y arquitectura.

```
rgap@ubuntu:~$ whoami
rgap
rgap@ubuntu:~$ hostname
ubuntu
rgap@ubuntu:~$ uname -a
Linux ubuntu 6.8.0 x86_64 GNU/Linux
```

Resultado: ya puedes demostrar quién usa la MV, cómo se llama y qué kernel ejecuta.

1. Ejecuta `whoami`, `hostname` y `cat /etc/os-release`.
2. Crea primero `~/curso_cloud` y después `sesion03` con `mkdir`.

Resultado completo esperado

```
$ whoami
rgap
$ hostname
ubuntu
$ cat /etc/os-release
NAME="Ubuntu"
VERSION="24.04 LTS"
$ mkdir ~/curso_cloud
$ mkdir ~/curso_cloud/sesion03
```

Seguridad

Comprende cada comando y confirma la ruta. No uses `sudo` —ejecución con privilegios administrativos— ni comandos de borrado sin saber qué cambiarán.

1. Ingresa con `cd`.
2. Confirma la ruta con `pwd`.
3. Guarda `uname -a > evidencia.txt`.
4. Revisa el archivo con `cat evidencia.txt`.

Resultado completo esperado

```
$ cd ~/curso_cloud/sesion03
$ pwd
/home/rgap/curso_cloud/sesion03
$ uname -a > evidencia.txt
$ cat evidencia.txt
Linux ubuntu 6.8.0 x86_64 GNU/Linux
```

Comprobación: el archivo `evidencia.txt` existe en la ruta mostrada y contiene la salida de `uname -a`.

Interfaces de red

Definición inicial

Una **interfaz de red** es la parte que Ubuntu usa para comunicarse con otros equipos.

Cómo reconocerla

Ubuntu le asigna un nombre. En nuestra MV veremos `ens160`.

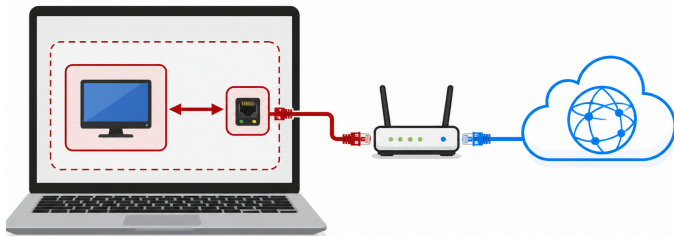
Qué ocurre por ella

Por la interfaz salen solicitudes y regresan respuestas.

Ejemplo sencillo

Al abrir un sitio web, la solicitud sale de Ubuntu por `ens160` y la respuesta regresa por esa misma interfaz.

El adaptador virtual enlaza la MV con la red externa



1. La MV usa su interfaz.
2. El adaptador virtual entrega el tráfico al hipervisor.
3. El router permite salir de la red local.
4. La conexión alcanza internet.

Líneas principales del resultado

```
rgap@ubuntu:~$ ip a
1: lo
   inet 127.0.0.1/8 scope host lo
2: ens160
   inet 192.168.35.151/24
   scope global dynamic ens160
```

`ip a` Lista interfaces y direcciones.

`lo` Interfaz interna del propio Ubuntu.

`ens160` Conecta la MV con la red.

`inet` Presenta una dirección IPv4: cuatro números separados por puntos.

`/24` Indica la parte que identifica la red local.

Qué significa este resultado

state UP: ens160 está activa. dynamic: su dirección fue asignada automáticamente. UNKNOWN en lo no es un error: no hay una conexión externa que comprobar porque lo funciona dentro de Ubuntu y no depende de un cable ni del adaptador virtual.

Otros campos: link/ether = dirección del adaptador; mtu, qdisc y qlen = transmisión de datos; inet6 = dirección IP más larga; valid_lft = tiempo de validez.

Una es interna y la otra conecta con la red

El resultado de `ip a` presenta dos interfaces con funciones diferentes.

`lo`: comunicación interna

Dirección: `127.0.0.1/8`.

Ubuntu la usa para comunicarse consigo mismo. No conecta la MV con otros equipos.

`ens160`: comunicación con la red

Dirección: `192.168.35.151/24`.

Es la interfaz que permite que la MV se comunique con la red externa.

Lectura correcta del resultado

Aunque `ip a` muestra dos interfaces, solo `ens160` conecta la MV con la red externa. `lo` funciona únicamente dentro de Ubuntu.

Ruta Regla que indica por dónde debe enviarse el tráfico hacia un destino.

Puerta de enlace Router que permite salir de la red local; también se denomina *gateway*.

DNS Servicio que traduce nombres, como `example.com`, a direcciones IP.

Ejemplo completo

En este laboratorio, la MV `192.168.35.151/24` sale mediante *Share with my Mac*. La Mac usa `192.168.100.140/24` en el Wi-Fi y su puerta de enlace es `192.168.100.1`. El DNS resuelve el nombre del sitio antes de iniciar la conexión.

NAT conecta las dos redes

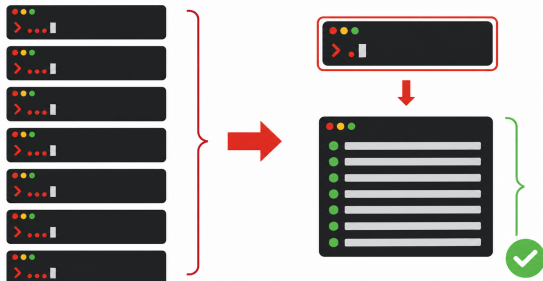
NAT significa traducción de direcciones de red. En *Share with my Mac*, la Mac recibe el tráfico de la red virtual y lo envía por su conexión Wi-Fi usando su propia dirección.



Flujo completo

El DNS convierte el nombre del sitio en una IP. La solicitud sale por `ens160`, la Mac aplica NAT, el Wi-Fi la entrega al router y el router la envía a internet. La respuesta regresa por el camino inverso hasta la MV.

La captura no muestra el servidor DNS ni la dirección pública utilizada por el router; por eso no se asignan valores a esos datos.



Alias

Nombre corto que representa **un comando** con sus opciones. Ejemplo: o puede representar `xdg-open ..`

Función

Nombre que agrupa **uno o varios comandos** ejecutados en orden. Forma: `nombre() { comandos; }`.

Duración en esta práctica

Ambos existen solo en la terminal actual. Al cerrarla, desaparecen.

Bash y Zsh son shells; \$SHELL indica la configurada

Variable de entorno

Dato con nombre que la shell conserva para describir la sesión. SHELL guarda la ubicación completa de la shell configurada. El comando `echo` muestra un texto o el valor de una variable.

Bash en Ubuntu

```
rgap@ubuntu:~$ echo $SHELL  
/bin/bash
```

Zsh en otra cuenta

```
rel@equipo:~$ echo $SHELL  
/bin/zsh
```

Resultado: ambas son shells; la salida permite reconocer cuál está configurada.

Dos elementos antes de crear el alias

xdg-open solicita a Ubuntu abrir un elemento con la aplicación gráfica predeterminada. El punto . representa el directorio actual.

Comando completo

```
rgap@ubuntu:~$ pwd  
/home/rgap  
rgap@ubuntu:~$ xdg-open .
```

Resultado visible

Ubuntu abre la aplicación **Archivos** y muestra el contenido de /home/rgap, que era el directorio actual.

Idea del alias: escribir o será más rápido que repetir xdg-open ..

1. Confirma la shell con `echo $SHELL`.
2. Crea alias `o='xdg-open .'`.
3. Ejecuta `o` y confirma que se abra **Archivos**.
4. Crea `info() { whoami; hostname; pwd; }`.
5. Ejecuta `info`.

Resultado completo esperado

```
$ echo $SHELL
/bin/bash
$ alias o='xdg-open .'
$ o
Se abre la carpeta actual en Archivos
$ info() { whoami; hostname; pwd; }
$ info
rgap
ubuntu
/home/rgap/curso_cloud/sesion03
```

Comprobación final

`o` abre la carpeta actual en Archivos; `info` devuelve usuario, equipo y ruta, en ese orden.

Archivo de configuración de Bash

.bashrc es un archivo oculto ubicado en el directorio personal. Bash lee sus instrucciones cada vez que inicia una terminal interactiva.

Solo en la terminal actual

Si creas o `echo` directamente en la terminal, desaparecen cuando la cierras.

Guardados en .bashrc

Si escribes allí el alias y la función, Bash vuelve a crearlos al abrir cada terminal nueva.

Dos comandos que utilizaremos

`nano ~/.bashrc` abre el archivo en un editor de texto. `source ~/.bashrc` vuelve a leerlo sin cerrar la terminal actual.

1. Abre el archivo con `nano ~/.bashrc`.
2. Agrega el alias `o` y la función `info` al final.
3. Guarda con `Ctrl+O`, confirma con `Enter` y sal con `Ctrl+X`.
4. Ejecuta `source ~/.bashrc`.
5. Prueba `o` e `info`.

Contenido y resultado esperado

```
$ nano ~/.bashrc
alias o='xdg-open .'
info() { whoami; hostname; pwd; }
$ source ~/.bashrc
$ o
Se abre la carpeta actual en Archivos
$ info
rgap
ubuntu
/home/rgap/curso_cloud/sesion03
```

Comprobación: abre una terminal nueva y ejecuta `o` e `info`; ambos deben seguir disponibles.



gracias



Universidad
Tecnológica
del Perú

- ▶ Debian Administrator's Handbook y páginas de manual: `bash`, `pwd`, `ls`, `mkdir`, `cd`, `cat`, `uname`, `ip`, `xdg-open` y `nano`.
- ▶ Documentación del hipervisor utilizado: límites de hardware virtual y modos de red disponibles.
- ▶ Material base proporcionado: *Fundamentos de Linux — Programa completo*.
- ▶ Ilustraciones educativas generadas para esta sesión: capas de Linux, flujo de terminal, sistema de archivos, red virtual y alias/función.